

INDUSTRIAL INTERNSHIP REPORT

on

PASSWORD MANAGER

Prepared by

Rokalla Lavanya

23981A05N4

Raghu Engineering College

Industrial Internship Programme

upskill Campus × The IoT Academy × UniConverge Technologies Pvt Ltd (UCT)

Academic Year: 2026

Executive Summary

This report presents the work completed during the industrial internship on a Python-based Password Manager application. The project focuses on providing a desktop graphical user interface for securely storing, retrieving, searching and deleting website account credentials.

The application is implemented using Python and Tkinter for the graphical user interface, SQLite for local data storage, hashlib for hashing the master password, the secrets module for password generation, and Fernet-based cryptography for encrypting stored account passwords.

The project includes user registration and login, master-password validation, password generation, encrypted password storage, password viewing after authentication, search functionality, deletion of stored credentials, and logout. The implementation also keeps stored passwords masked in the dashboard table.

The internship report follows the structure of the supplied internship report template and adapts its sections to the Password Manager project. Claims about actual performance measurements or screenshots are intentionally left for verification from the student's own execution environment.

Table of Contents

1 Preface	3
2 Introduction	4
2.1 About UniConverge Technologies Pvt Ltd	4
2.2 About upskill Campus	5
2.3 The IoT Academy	5
2.4 Objectives of the Internship Program	6
2.5 Reference	6
2.6 Glossary	6
3 Problem Statement	7
4 Existing and Proposed Solution	8
4.1 Code Submission (GitHub Link)	9
4.2 Report Submission (GitHub Link)	9
5 Proposed Design / Model	10
5.1 High-Level Design	10
5.2 Low-Level Design	11
5.3 Interfaces	12
6 Performance Test	13
6.1 Test Plan / Test Cases	13
6.2 Test Procedure	14
6.3 Performance Outcome	14
7 My Learnings	15
8 Future Work Scope	16

1 Preface

The six-week industrial internship provided an opportunity to apply programming concepts to a practical software project. The assigned work resulted in the development of a Password Manager desktop application using Python.

The project addresses a common digital-security requirement: users often maintain credentials for multiple websites and applications. A password manager can reduce the need to remember every credential separately while providing a central interface for managing stored information.

The internship also provided practical exposure to GUI development, database operations, input validation, authentication logic, password hashing, encryption/decryption, exception handling and project documentation.

The project was planned around understanding the requirements, implementing the database and security logic, developing the graphical interface, integrating the modules, testing the major functions and documenting the final solution.

The internship report template supplied for the programme asks for a preface, introduction, problem statement, proposed solution, design/model, testing, learnings and future scope. This report follows that organization while describing the actual Password Manager implementation.

Acknowledgement: I sincerely thank the internship coordinators, mentors, faculty members and everyone who supported me during the internship and helped me gain practical experience.

Message to juniors and peers: Use internship projects as an opportunity to understand how individual programming concepts become part of a complete working application. Keep the code organized, test each feature and document the work while developing it.

2 Introduction

The Password Manager is a desktop application intended to help a user organize website credentials in a local SQLite database. It provides a graphical interface rather than requiring command-line interaction.

2.1 About UniConverge Technologies Pvt Ltd

The supplied internship template describes UniConverge Technologies Pvt Ltd (UCT) as a company established in 2013 and working in the digital transformation domain, with a focus on sustainability and return on investment. The template lists technologies and areas including IoT, cyber security, cloud computing, Java Full Stack, Python, front-end technologies and communication technologies.

¶fileciteÖturn3file0ÖL54-L67Ô

The template also describes UCT offerings and solution areas such as IoT platforms, smart factory solutions, LoRaWAN-based solutions and predictive maintenance. ¶fileciteÖturn3file0ÖL71-L107Ô

2.2 About upskill Campus (USC)

According to the supplied report template, upskill Campus, together with The IoT Academy and in association with UniConverge Technologies, facilitated the internship process. The template describes USC as a career development platform. ¶fileciteÖturn3file0ÖL103-L107Ô

2.3 The IoT Academy

The supplied template describes The IoT Academy as the EdTech division of UCT and states that it runs long executive certification programmes in collaboration with EICT Academy and IIT institutions in multiple domains. ¶fileciteÖturn3file0ÖL114-L116Ô

2.4 Objectives of the Internship Program

The internship template identifies practical industry experience, solving real-world problems, improving job prospects, understanding the field and its applications, and personal growth in communication and problem solving as key objectives.

For this project, these objectives were applied through the following technical goals:

- Develop a functional desktop Password Manager using Python.
- Implement user registration and login with master-password validation.
- Store application data using SQLite.
- Hash the master password before storing it in the database.
- Encrypt stored account passwords using Fernet.
- Generate strong random passwords using the Python secrets module.
- Provide search, view, delete and logout functionality.
- Document the design, testing approach, learning outcomes and future improvements.

2.5 Reference

1. Supplied Industrial Internship Report template from the internship programme.
2. Project source file: password_manager.py.
3. Python standard library modules and the cryptography package used by the implementation.

2.6 Glossary

Term	Meaning
GUI	Graphical User Interface
SQLite	Lightweight relational database engine
Hashing	One-way transformation used here for the master password
Encryption	Transformation of data into protected ciphertext
Fernet	Authenticated symmetric encryption mechanism used by the project
Tkinter	Python GUI toolkit used by the application
CRUD	Create, Read, Update and Delete style data operations

3 Problem Statement

Managing credentials for multiple websites can become difficult when users rely on memory, notes or scattered storage locations. A practical password-management application should provide a convenient way to save account details while reducing direct exposure of stored passwords.

The problem addressed by this project is the design and implementation of a simple desktop Password Manager that authenticates a user with a master password, stores account records in a local database, protects stored account passwords through encryption, and provides controlled retrieval and management through a graphical interface.

The project therefore combines three main requirements: usability, persistent local storage and protection of stored credentials.

Functional Requirements

- Register a new user with a username and master password.
- Validate that the master password meets the implemented minimum length.
- Authenticate an existing user during login.
- Generate a 16-character password using letters, digits and selected special characters.
- Add website, account username and password records.
- Encrypt account passwords before database storage.
- Display saved records with the password masked.
- Search records by website or account username.
- View the decrypted password for a selected record after login.
- Delete a selected credential and log out.

4 Existing and Proposed Solution

Existing Approaches

Common approaches to credential management include remembering passwords, storing them in plain text notes, or using dedicated password-management software. Plain-text storage can expose credentials if the storage is accessed by another person or application. Manual management can also become difficult as the number of accounts increases.

This project is intentionally a compact educational implementation rather than a replacement for mature production password-management systems. Its purpose is to demonstrate authentication, database persistence, hashing, encryption and GUI development in one application.

Proposed Solution

The proposed solution is a local Python desktop Password Manager. A user first registers and then logs in using the master password. The master password is hashed using SHA-256 for database comparison. A deterministic key derived from the master password is used to construct a Fernet cipher. Individual account passwords are encrypted before being inserted into SQLite.

After successful login, the dashboard allows the authenticated user to add credentials, generate passwords, search records, view a selected password, delete records and log out.

Value Addition

- Combines GUI, database and security concepts in one application.
- Keeps stored account passwords encrypted rather than directly storing plaintext account passwords.
- Masks password values in the dashboard table.
- Provides a built-in password generator.
- Uses parameterized SQL queries for database operations.

4.1 Code Submission (GitHub Link)

GitHub repository: [ADD YOUR GITHUB REPOSITORY LINK AFTER CREATING THE REPOSITORY]

Expected repository structure:

```
upskillcampus/  
├── README.md  
├── password_manager.py  
└── PasswordManager_Internship_Report.pdf
```

4.2 Report Submission (GitHub Link)

Report PDF link: [ADD YOUR GITHUB REPORT PDF LINK AFTER UPLOADING THE PDF]

The internship template specifically asks for a code submission link and a report submission link in the report.

Important: Replace the placeholders above with the actual GitHub links after the repository and PDF have been uploaded.

5 Proposed Design / Model

The Password Manager follows a simple layered flow: user interface → validation/authentication → database and cryptographic operations → dashboard operations. The implementation is contained in a Python source file and uses SQLite as the local persistent store.

5.1 High-Level Diagram

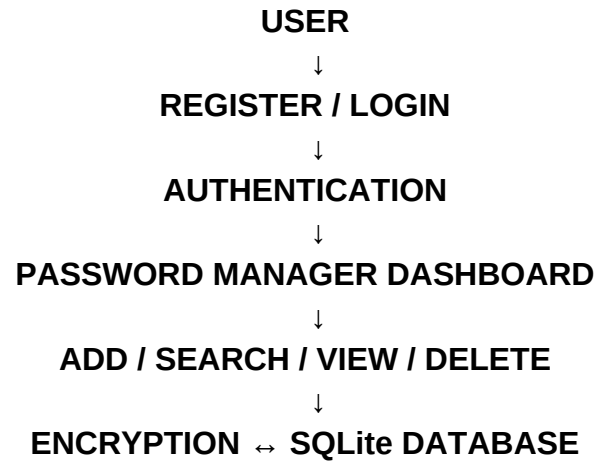


Figure: High-level flow of the Password Manager

Suggested diagram content: User → Login/Register → Authentication → Password Manager Dashboard → Add/Search/View/Delete → SQLite Database. The encryption/decryption module should be shown between the dashboard operations and stored password data.

5.2 Low-Level Diagram

The main implementation modules/functions are organized around the following responsibilities:

Module / Function	Responsibility
hash_master_password()	Creates SHA-256 hash of the master password.
create_encryption_key()	Derives a Fernet-compatible key from the master password.
register_user()	Validates registration input and creates the user record.
login_user()	Authenticates username/master password and initializes the cipher.
encrypt_password()	Encrypts an account password before storage.
decrypt_password()	Decrypts an account password for controlled viewing.
generate_password()	Generates a 16-character random password.
add_password()	Validates and saves a credential record.
load_passwords()	Loads the current user's saved records into the table.
search_passwords()	Filters records by website or account username.
view_password()	Retrieves and decrypts a selected password.
delete_password()	Deletes a selected credential after confirmation.
logout()	Clears current-user and cipher state and returns to login.

5.3 Interfaces

The application uses Tkinter widgets to provide the graphical interface. The source code defines login, registration and dashboard screens.

Login / Registration Interface

Figure: Password Manager login interface

Insert a screenshot of the running login screen here. The implementation contains username and master-password fields and a path to registration.

Dashboard Interface

Figure: Password Manager dashboard with a saved credential

A screenshot of the running dashboard is included above. The dashboard shows the authenticated user, credential entry fields, password generator, search controls, a credential table and action controls.

Figure: Controlled viewing of a selected password after login

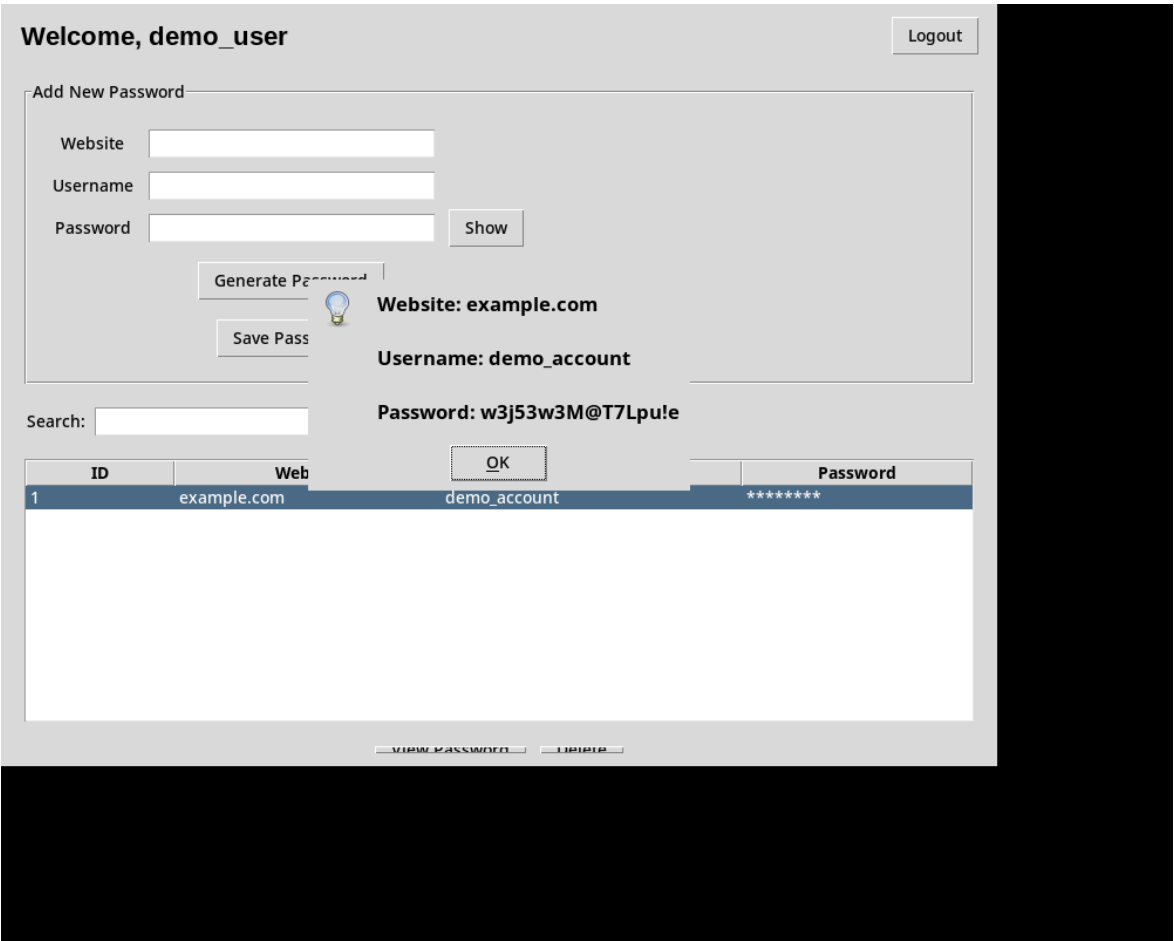


Figure: Search functionality showing matching credential records

Welcome, demo_user

Logout

Add New Password

Website

Username

Password

Show

Generate Password

Save Password

Clear

Search:

example

Search

Clear

ID	Website	Username	Password
1	example.com	demo_account	*****

VIEW PASSWORD

HIDE PASSWORD

Data Model

The project creates two SQLite tables:

- users: id, username, master_password and encryption_key.
- passwords: id, username, website, account_username and encrypted_password.

The password table associates each stored credential with the logged-in username, while the password itself is stored in encrypted form.

6 Performance Test

The supplied internship template emphasizes constraints, test results and the relationship between testing and real-world use.

For this project, the primary test focus is functional correctness, data persistence, authentication, encryption/decryption behavior and handling of invalid input. No numerical benchmark should be claimed unless it is measured during execution.

6.1 Test Plan / Test Cases

Test Case	Input / Action	Expected Result	Status
TC01	Register with valid username and master password	Account is created and login screen is shown	Verify
TC02	Register with mismatched passwords	Error message is shown; account is not created	Verify
TC03	Register with master password shorter than 8 characters	Weak-password warning is shown	Verify
TC04	Login with valid credentials	Dashboard opens	Verify
TC05	Login with invalid credentials	Login failure message is shown	Verify
TC06	Generate password	A 16-character generated password appears	Verify
TC07	Add website credential	Credential is encrypted and saved	Verify
TC08	Search saved credential	Matching website/account records are displayed	Verify
TC09	View selected password	Selected password is decrypted and shown	Verify
TC10	Delete selected credential	Credential is removed after confirmation	Verify
TC11	Logout	Session state is cleared and login screen returns	Verify

6.2 Test Procedure

4. Start the Python application and confirm that the Password Manager window opens.
5. Create a test user through the registration screen.
6. Attempt invalid registration cases such as empty fields, mismatched passwords and a password shorter than eight characters.
7. Log in with valid and invalid credentials and observe the application response.
8. Generate a password and confirm that the generated value is inserted into the password field.
9. Add a test website credential and confirm that the record appears in the dashboard with its password masked.
10. Use the search field to locate a saved website or account username.
11. Select a record and use View Password to confirm controlled decryption.
12. Delete a test record and confirm that it is removed from the displayed list.
13. Log out and confirm that the application returns to the login screen.

The application was run successfully in a desktop-style test environment for this report. The screenshots use demonstration data (demo_user, example.com and demo_account), not the student's personal credentials. The test-case status should still be updated by the student if additional tests are performed locally.

6.3 Performance Outcome

From the implementation, the application is designed as a lightweight local desktop application using SQLite and Tkinter. The expected operational workload is small-scale personal credential management. The source code does not include formal timing, memory, throughput or stress-test measurements; therefore this report does not claim numerical performance results.

For the final submission, actual observations such as startup time, response time during search, number of test records used and any errors found can be added after execution.

7 My Learnings

The Password Manager project provided practical learning across several areas of software development.

- Python application development: improved understanding of functions, modules, exception handling and program flow.
- GUI development: learned how Tkinter widgets, frames, buttons, entry fields and Treeview components can be combined into a desktop interface.
- Database programming: gained practical experience creating SQLite tables, inserting records, selecting records, filtering data and deleting records.
- Authentication: understood the role of input validation and password verification in a login workflow.
- Hashing and encryption: learned the difference between storing a hash for authentication and encrypting data that must later be recovered.
- Security-aware programming: implemented masked password display, encrypted credential storage and generated passwords.
- Testing: learned to convert individual features into test cases and verify both successful and invalid-input scenarios.
- Project documentation: learned how to describe a software project using problem statements, design, testing, learning outcomes and future scope.

These skills are useful for future software-development projects because they connect programming concepts with a complete application workflow.

8 Future Work Scope

The current implementation is a compact educational Password Manager. Several improvements could be considered in a future version:

- Use a stronger password-hashing design with a dedicated password KDF such as Argon2, scrypt or PBKDF2 and a unique salt for each user.
- Separate the encryption key-management design from the master-password hashing design.
- Add password update/edit functionality.
- Add import/export with a carefully designed encrypted backup format.
- Add automatic locking after inactivity.
- Add configurable password-generation rules and strength indicators.
- Improve the visual design and accessibility of the Tkinter interface.
- Add more comprehensive automated tests and error logging.
- Consider secure memory handling and key lifecycle improvements.
- For a production-grade system, perform a dedicated security review and threat-model assessment.

These are future possibilities rather than features claimed to be present in the current source code.

Final Conclusion

The Password Manager demonstrates how Python, Tkinter, SQLite, hashing and encryption can be integrated into a single desktop application. The project addresses a practical credential-management problem and provides authentication, encrypted storage, password generation and credential-management functions.

The internship work helped connect programming fundamentals with software design, database management, security concepts, testing and technical documentation. The project can serve as a foundation for a more comprehensive password-management system with stronger key management, expanded testing and additional security controls.

Final Submission Checklist

- Replace Rokalla Lavanya, [ROLL NUMBER] and Raghu Engineering College.
- Run password_manager.py and insert actual screenshots in the marked places.
- Execute the test cases and replace 'Verify' with the actual result/status.
- Create your GitHub repository and upload password_manager.py.
- Upload this report as a PDF to the same repository.
- Replace the GitHub placeholders with your real repository/report links.
- Export this DOCX to PDF before final submission if the internship portal requires PDF.

Source/template note: This report was adapted from the supplied 16-page Industrial Internship Report template, whose structure includes Preface, Introduction, Problem Statement, Existing and Proposed Solution, Proposed Design/Model, Performance Test, Learnings and Future Work Scope.

file:///C:/Users/turn3/Desktop/Industrial%20Internship%20Report%20Template.docx

Prepared for internship submission — verify all placeholders and execution results before submitting.